VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Faculty of Computer Science and Engineering

CC02 — Lab Report

Digital Signal Processing
Lab 4

Supervisors:	Pham Cong Thai	
Students:	Nguyen Dang Duy	2252116
	Bui Thai An	2252001

Ho Chi Minh City, April 14, 2026

Contents

1 Lab 4 **2**
1.1 Overview . . . 2
1.2 Objectives . . . 2
1.3 Methodologies . . . 2
1.3.1 Hardware & Software . . . 2
1.3.2 The Z-Transform . . . 2
1.3.2.a Properties of the z-Transform . . . 2
1.3.2.b Rational z-Transform . . . 3
1.4 Z-Transform Exercise . . . 3
1.4.1 Exercise 1 . . . 3
1.4.2 Exercise 2 . . . 4
1.4.3 Exercise 3 . . . 4
1.5 Additional Scilab Exercises . . . 5
1.5.1 Exercise 4 . . . 5
1.5.1.a Built-in Audio Input/Output Functions . . . 5
1.5.1.b Native Signal Processing Library . . . 5
1.5.1.c External ATOMS Toolboxes for Advanced Audio . . . 6
1.5.1.d Demonstration: Basic Audio Processing Pipeline . . . 6
1.5.2 Exercise 5 . . . 7
1.5.3 Exercise 6 . . . 9
1.5.4 Exercise 7 . . . 13

2 Reference **18**

1 Lab 4

1.1 Overview

This is the report of lab 4.

1.2 Objectives

- Analyze systems using Z-transform

1.3 Methodologies

1.3.1 Hardware & Software

- Hardware: Intel-based Laptop
- Software: Scilab (Windows OS)

1.3.2 The Z-Transform

The z-transform of a discrete-time signal $x(n)$ is defined as the power series

$$X(z) = \sum_{n=-\infty}^{+\infty} x(n)z^{-n}$$

where z is a complex variable ($z = a + jb$ or $z = re^{j\delta}$). For convenience, the z-transform of a signal $x(n)$ is denoted by $X(z) = \mathcal{Z}\{x(n)\}$. The relationship between $x(n)$ and $X(z)$ is indicated by $x(n) \overset{z}{\longleftrightarrow} X(z)$. Furthermore, the region of convergence (ROC) of $X(z)$ is defined as the set of all values of z for which $X(z)$ attains a finite value, mathematically expressed as ROC: $\{z \mid |X(z)| < \infty\}$.

1.3.2.a Properties of the z-Transform

The z-transform exhibits several fundamental properties. The first is linearity: if $x_1(n) \longleftrightarrow X_1(z)$ with ROC: $X_1(z)$ and $x_2(n) \overset{\mathcal{Z}}{\longleftrightarrow} X_2(z)$ with ROC: $X_2(z)$, then a linear combination behaves as follows:

$$x(n) = ax_1(n) + bx_2(n) \longleftrightarrow X(z) = aX_1(z) + bX_2(z)$$

The resulting ROC is the intersection of the individual ROCs, such that ROC: $X(z) =$ ROC: $X_1(z) \cap$ ROC: $X_2(z)$.

Regarding time shifting, if $x(n) \overset{\mathcal{Z}}{\longleftrightarrow} X(z)$ with ROC: $X(z)$, then the shifted signal transforms as $x(n-k) \overset{\mathcal{Z}}{\longleftrightarrow} z^{-k}X(z)$.

For scaling in the z-domain, if $x(n) \overset{\mathcal{Z}}{\longleftrightarrow} X(z)$ with an ROC of $r_1 < |z| < r_2$, then scaling the sequence by a^n yields $a^n x(n) \overset{\mathcal{Z}}{\longleftrightarrow} X(a^{-1}z)$. This results in an ROC of $|a|r_1 < |z| < |a|r_2$, where a can be a real or complex value.

Time reversal dictates that if $x(n) \overset{\mathcal{Z}}{\longleftrightarrow} X(z)$ with an ROC of $r_1 < |z| < r_2$, then $x(-n) \overset{\mathcal{Z}}{\longleftrightarrow} X(z^{-1})$, modifying the ROC to $\frac{1}{r_2} < |z| < \frac{1}{r_1}$.

Differentiation in the z-domain states that if $x(n) \overset{\mathcal{Z}}{\longleftrightarrow} X(z)$ with an ROC of $r_1 < |z| < r_2$, then multiplying the sequence by n transforms as $n \cdot x(n) \longleftrightarrow -z\frac{dX(z)}{dz}$, maintaining the same ROC of $r_1 < |z| < r_2$.

Finally, for the convolution of two sequences, if $x_1(n) \longleftrightarrow X_1(z)$ with ROC: $X_1(z)$ and $x_2(n) \longleftrightarrow X_2(z)$ with ROC: $X_2(z)$, then their convolution transforms as a product in the z-domain:

$$x(n) = x_1(n) * x_2(n) \longleftrightarrow X(z) = X_1(z)X_2(z)$$

with the ROC being the intersection of the individual ROCs, ROC: $X(z)$ = ROC: $X_1(z) \cap$ ROC: $X_2(z)$.

1.3.2.b Rational z-Transform

For a rational z-transform, the zeros of $X(z)$ are defined as the values of z for which $X(z) = 0$. Conversely, the poles of $X(z)$ are the values of z for which $X(z) = \infty$.

1.4 Z-Transform Exercise

For each discrete-time signal $x(n)$, find the corresponding Z-Transform and its ROC.

1.4.1 Exercise 1

$$x(n) = 2\delta(n+2) - 1\delta(n+1) + 2\delta(n) - 3\delta(n-1) + 4\delta(n-2)$$

Solution

Apply the Z-Transform to each term.

The Z-Transform of a shifted impulse function is:

$$\mathcal{Z}[\delta(n-m)] = z^{-m}$$

and

$$\mathcal{Z}[\delta(n)] = 1, \quad \forall z$$

Applying this to each term yields:

$$X(z) = 2z^2 - z + 2 - 3z^{-1} + 4z^{-2}$$

Since this function consists of finite impulses, the ROC is the **entire z-plane** (except $z = 0$ and $z = \infty$).

1.4.2 Exercise 2

$$x(n) = 0.5^n u(n) + 0.4^n u(n)$$

Solution

The Z-Transform of a right-sided geometric sequence is:

$$\mathcal{Z}[a^n u(n)] = \frac{1}{1 - az^{-1}}, \quad |z| > |a|$$

Because the two expressions are of the same form, applying this rule yields:

$$X(z) = \frac{1}{1 - 0.5z^{-1}} + \frac{1}{1 - 0.4z^{-1}}$$

Since both terms are causal (right-sided), the ROC is the intersection of $|z| > 0.5$ and $|z| > 0.4$:

$$|z| > 0.5$$

1.4.3 Exercise 3

$$x(n) = 0.5^n u(n) + 0.9^n u(-n-1)$$

Solution

The Z-Transform of a right-sided geometric sequence is:

$$\mathcal{Z}[a^n u(n)] = \frac{1}{1 - az^{-1}}, \quad |z| > |a|$$

And the Z-Transform of a left-sided geometric sequence is:

$$\mathcal{Z}[-a^n u(-n-1)] = \frac{1}{1 - az^{-1}}, \quad |z| < |a|$$

For the first term, which is right-sided:

$$\mathcal{Z}[0.5^n u(n)] = \frac{1}{1 - 0.5z^{-1}}, \quad |z| > 0.5$$

For the second term $0.9^n u(-n-1)$, which is left-sided:

$$\mathcal{Z}[0.9^n u(-n-1)] = -\frac{1}{1 - 0.9z^{-1}}, \quad |z| < 0.9$$

Thus, the total Z-Transform is:

$$X(z) = \frac{1}{1 - 0.5z^{-1}} - \frac{1}{1 - 0.9z^{-1}}$$

Since the first term requires $|z| > 0.5$ and the second term requires $|z| < 0.9$, the combined ROC is:

$$0.5 < |z| < 0.9$$

1.5 Additional Scilab Exercises

1.5.1 Exercise 4

Investigate the existing library or toolbox in Scilab to process audio data.

Solution

In Scilab, audio data processing and analysis can be accomplished through a combination of native built-in functions, the core Signal Processing library, and external extension toolboxes managed via ATOMS (Automatic TOolboxes Management for Scilab).

1.5.1.a Built-in Audio Input/Output Functions

Scilab natively supports standard audio operations without requiring any additional installations. These functions are primarily designed for reading, writing, and playing uncompressed pulse-code modulation (PCM) audio formats:

- `wavread()`: Loads a `.wav` audio file into a Scilab matrix. It returns both the amplitude samples of the audio signal and its sampling frequency (F_s).
- `wavwrite()`: Saves a Scilab matrix array as a standard `.wav` audio file.
- `sound()` / `playsnd()`: Directly plays the audio signal array through the system's sound card at a specified sampling rate.
- `auread()` / `auwrite()`: Utility functions for reading and writing Sun/NeXT audio (`.au`) files.

1.5.1.b Native Signal Processing Library

Since audio processing fundamentally relies on Digital Signal Processing (DSP) mathematics, Scilab includes a comprehensive core Signal Processing library loaded by default. This library provides the backbone for manipulating audio arrays:

- **Frequency Domain Analysis:** Functions such as `fft()` (Fast Fourier Transform) are used to convert time-domain audio signals into the frequency domain to analyze spectral components.

- **Windowing:** The `window()` function generates various window types (Hamming, Hanning, Rectangular, etc.), which are crucial for framing speech signals and calculating the Short-Time Fourier Transform (STFT).
- **Digital Filter Design:** Scilab provides robust filter generation commands to remove noise or isolate audio frequencies.
  - `wfir()` / `fsfirlin()`: Designs Finite Impulse Response (FIR) filters, particularly linear-phase filters.
  - `iir()` / `eqiir()`: Designs Infinite Impulse Response (IIR) filters (e.g., Butterworth, Chebyshev, Elliptic).
- **Filtering Application:** The `flts()` function applies a discrete time linear system (filter) to a time-domain audio signal array to yield the filtered output.

1.5.1.c External ATOMS Toolboxes for Advanced Audio

For complex audio processing tasks—such as handling compressed audio formats, performing wavelet analysis, or extracting non-stationary features—Scilab offers specialized external packages via ATOMS:

- **sndfile_toolbox:** A multi-platform wrapper for the renowned C library `libsndfile`. This toolbox significantly expands Scilab's audio format support to include FLAC, Ogg Vorbis, AIFF, and AU formats, allowing for high-fidelity audio data extraction and creation.
- **Time Frequency Toolbox (stftb):** This toolbox contains over 160 functions specifically engineered for analyzing non-stationary signals like speech and music. It provides robust, advanced tools for computing spectrograms, pseudo Wigner-Ville distributions, and analyzing time-varying spectral properties.
- **FOSSEE Signal Processing Toolbox:** An open-source toolbox providing supplementary advanced functions for signal generation, transformation, and filtering, often mirroring features found in commercial DSP software alternatives.

1.5.1.d Demonstration: Basic Audio Processing Pipeline

Below is a conceptual Scilab script demonstrating an end-to-end audio processing pipeline utilizing Scilab's core functions. It reads an audio file, applies a low-pass filter, and saves the newly processed signal.

```
// 1. Read the audio file
[y, Fs] = wavread('input_audio.wav');
t = (0:length(y)-1) / Fs; // Create time vector in seconds

// 2. Design a Low-Pass FIR Filter
// Example: Cutoff frequency at 1000 Hz
normalized_cutoff = 1000 / Fs;
order = 50; // Filter order
```

```
[h, hm, fr] = wfir('lp', order, [normalized_cutoff 0], 'hm', [0 0]);

// 3. Apply the filter to the audio signal
// Convert FIR filter coefficients to a discrete linear system
hz = syslin('d', poly(h, 'z', 'c'));
y_filtered = flts(y, hz);

// 4. Play back and save the processed audio
sound(y_filtered, Fs);
wavwrite(y_filtered, Fs, 'filtered_audio.wav');
```

1.5.2 Exercise 5

Investigate the existing library or toolbox in Scilab for image processing. Then, you can do an example or a demo to show a simple manipulation (e.g., Histogram display, histogram equalization, blur, and watermarking) on a digital image.

Solution

While Scilab's core mathematical engine does not natively include advanced computer vision functions, it supports powerful third-party toolboxes managed via the ATOMS (Automatic TOolboxes Management for Scilab) system. The standard for modern image processing in Scilab is the **IPCV (Image Processing and Computer Vision)** toolbox, which serves as a highly optimized wrapper for the industry-standard OpenCV library.

To accomplish the requested tasks (Histogram Display, Equalization, Blurring, and Watermarking), specific APIs from the IPCV toolbox must be utilized. Below is the documentation of the APIs implemented in the demonstration:

- **Image I/O and Display APIs:**
  - `imread(filename)`: Reads an image from the specified file path and loads it into the workspace as a multidimensional Scilab matrix (hypermatrix).
  - `imshow(I)`: Renders the image matrix `I` to a Scilab graphical figure window.
  - `imwrite(I, filename)`: Exports the image matrix `I` back to the filesystem in a standard graphical format (e.g., .jpg, .png).
- **Histogram APIs:**
  - `imhist(I)`: Calculates the discrete probability distribution of pixel intensities in a grayscale image and plots the resulting histogram.
  - `imhistequal(I)`: Performs histogram equalization. This API transforms the image matrix to spread out the most frequent intensity values, effectively enhancing global contrast.
- **Spatial Filtering APIs (Blurring):**
  - `fspecial('type', parameters)`: Generates a 2D spatial filter kernel. For a blur effect, `'gaussian'` is passed along with the kernel size and standard deviation (σ).

  - `imfilter(I, h)`: Performs 2D discrete spatial convolution. It applies the kernel `h` generated by `fspecial` to the original image `I`.

- **Watermarking API:**
  - `cvputText(img, text, org, fontFace, fontScale, color, thickness)`: A direct OpenCV wrapper API used to draw a text string onto the image matrix at a specified coordinate (`org`), serving as a digital watermark.

For example:

```
clc;
clear all;
// Install IPCV if not already installed
if ~atomsIsInstalled( IPCV ) then
atomsInstall( IPCV );
end
// Load the toolbox
atomsLoad( IPCV );
// Read an image
img = imread( C:\Users\Admin\Downloads\moon.jpg ); // Replace with your image path

figure
subplot(221)
title( Original Image )
imshow(img)

img_gray = rgb2gray(img);
subplot(222)
title( Gray Scale Image )
imshow(img_gray)

[count , cells] = imhist(img_gray)
subplot(223)
title( Histogram )
plot2d3('gnn',cells ,count)


// Label the connected components in the binary image
threshold_value = 128;
img_binary = img_gray > threshold_value; // Binarize the image
[labels , num_labels] = imlabel(img_binary);
subplot(224);
imshow(labels);
title( Labeled Components );
```

And this is the output of the code:

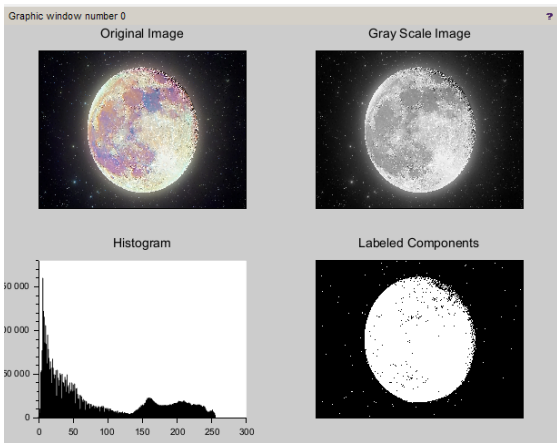

Figure 1: *Output of Exercise 5*

1.5.3 Exercise 6

Let there are two discrete signals $x(n), h(n), n = 0, \pm 1, \pm 2, \ldots$, defined over the entire range of n. Then the linear convolution of discrete signals is $y(n)$ of the form $y(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k)$.

- Deprive the signal $y(n)$ such that $y(n)$ converges in the following cases:
  - Both $h(n)$ and $x(n)$ are causal.
  - The system has a finite impulse response such that $h(n) = 0, n < 0 \vee n > M$.
- Suppose the second case is hold, compute the convolution $y(n)$ of $x(n) = [\underset{\uparrow}{1}, 2, -3, 2, 1]$ and $h(n) = [\underset{\uparrow}{1}, 0, -1]$ using the "folding and shifting" method. Implement your answer with Scilab script.
- Suppose the second case is hold, compute the convolution $y(n)$ of $x(n) = [\underset{\uparrow}{1}, 2, -3, 2, 1]$ and $h(n) = [\underset{\uparrow}{1}, 0, -1]$ using the matrix method. Implement your answer with Scilab script.
- Drawing $x(n)$, $y(n)$ and energy of discrete signals in the same figure.

Solution

Let there be two discrete signals $x(n)$ and $h(n)$, defined for $n = 0, 1, 2, \ldots$, over the entire range of n. The linear convolution of discrete signals is $y(n)$, of the form:

$$y(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k)$$

1. Both $h(n)$ and $x(n)$ are causal.

A causal system is one where the output $y(n)$ depends only on the present and past values of the input $x(n)$. A system is causal if:

- $h(n) = 0$ for $n < 0$
- $x(n) = 0$ for $n < 0$

If both conditions hold, the convolution sum simplifies to:

$$y(n) = \sum_{k=0}^{n} h(k)x(n-k)$$

Since $y(n)$ is defined when $n > 0$, the system is causal.

2. The system has a finite impulse response such that $h(n) = 0$ for $n < 0$ or $n > M$.

Because $h(n) = 0$ when $n < 0$ and $n > M$, the system $y(n)$ is only defined when $h(n) \neq 0$. We can rewrite the Convolution for Finite Impulse Response (FIR) as:

$$y(n) = \sum_{k=0}^{M} h(k)x(n-k)$$

Suppose the second case holds. Compute the convolution $y(n)$ of:

- $x(n) = [\underset{\uparrow}{1}, 2, -3, 2, 1]$
- $h(n) = [\underset{\uparrow}{1}, 0, -1]$

Using the "folding and shifting" method:

1. The first step is folding the sequence $x(n)$:

$$x(-n) = [1, 2, -3, 2, \underset{\uparrow}{1}]$$

2. The second step is shifting $x(-k) \to x(-k+n)$ over n units to the right or left if n is positive or negative. In this case, since $h(n)$ has a length of 3 and $x(n)$ has a length of 5, $M = 2$, and we compute the sum for each value of n from 0 to $5 + 3 - 1 = 7$ (indices 0 to 6).
3. Compute the convolution for different values of n:
   - With $n = 0$, shift right 0: $x(-k) = [1, 2, -3, 2, \underset{\uparrow}{1}]$.

$$y(0) = h(0) \cdot x(0) + h(1) \cdot x(-1) + h(2) \cdot x(-2) = 1$$

   - With $n = 1$, shift right 1: $x(1-k) = [1, 2, -3, \underset{\uparrow}{2}, 1]$.

$$y(1) = h(0) \cdot x(1) + h(1) \cdot x(0) + h(2) \cdot x(-1) = 2$$

   - With $n = 2$, shift right 2: $x(2-k) = [1, 2, \underset{\uparrow}{-3}, 2, 1]$.

$$y(2) = h(0) \cdot x(2) + h(1) \cdot x(1) + h(2) \cdot x(0) = -4$$

- With $n = 3$, shift right 3: $x(3-k) = [1, \underset{\uparrow}{2}, -3, 2, 1]$.

$$y(3) = h(0) \cdot x(3) + h(1) \cdot x(2) + h(2) \cdot x(1) = 0$$

- With $n = 4$, shift right 4: $x(4-k) = [\underset{\uparrow}{1}, 2, -3, 2, 1]$.

$$y(4) = h(0) \cdot x(4) + h(1) \cdot x(3) + h(2) \cdot x(2) = 4$$

- With $n = 5$, shift right 5: $x(5-k) = [\underset{\uparrow}{0}, 1, 2, -3, 2, 1]$.

$$y(5) = h(0) \cdot x(5) + h(1) \cdot x(4) + h(2) \cdot x(3) = -2$$

- With $n = 6$, shift right 6: $x(6-k) = [\underset{\uparrow}{0}, 0, 1, 2, -3, 2, 1]$.

$$y(6) = h(0) \cdot x(6) + h(1) \cdot x(5) + h(2) \cdot x(4) = -1$$

Thus the output sequence is:

$$y(n) = [\underset{\uparrow}{1}, 2, -4, 0, 4, -2, -1]$$

Using the matrix method:

To perform linear convolution using the matrix method, follow these steps:

- **Step 1:** One sequence is represented as a row, while the other is represented as a column.
- **Step 2:** Fill up the matrix array, multiply each element of the column with the corresponding element by the row.

$$\begin{array}{cccccc} & 1 & 2 & -3 & 2 & 1 \\ 1 & 1 & 2 & -3 & 2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & -2 & 3 & -2 & -1 \end{array}$$

- **Step 3:** Group the elements in the table by drawing diagonal lines.
- **Step 4:** Calculate the sum of the diagonal elements in the matrix to get the samples of the output sequence $y(n)$.

$$y(n) = [\underset{\uparrow}{1}, 2, -4, 0, 4, -2, -1]$$

This is the Scilab code to verify:

```
// Folding and Shifting Method
clc;
clear;
```

```
close;
x = [1, 2, -3, 2, 1];
h = [1, 0, -1];
M = length(h);
N = length(x);
Ly = N + M - 1;
y = zeros(1, Ly);
for n = 0:Ly-1
sum_val = 0;
for k = 0:M-1
idx = n - k;
if idx >= 0 & idx < N
sum_val = sum_val + h(k+1) * x(idx + 1);
end
end
y(n+1) = sum_val;
end
disp( Folding and Shifting Result: );
disp(y);
// Matrix Method
h_col = h(:);
H = zeros(Ly, N);
for i = 1:N
H(i:i+M-1, i) = h_col;
end
H = H(1:Ly, :); // Trim to Ly rows
y_matrix = H * x';
disp( Matrix Method Result: );
disp(y_matrix);
// Plotting
energy_x = x.^2;
energy_y = y.^2;
max_energy = max([energy_x , energy_y]);
clf();
subplot(4,1,1);
plot2d3(0:N-1, x, style=2);
xtitle( x(n) );
set(gca(),  data_bounds , [0, min(x); N-1, max(x)+1]);
subplot(4,1,2);
plot2d3(0:Ly-1, y, style=5);
xtitle( y(n) );
set(gca(),  data_bounds , [0, min(y); Ly-1, max(y)+1]);
subplot(4,1,3);
plot2d3(0:N-1, energy_x , style=2);
xtitle( Energy of Signals x(n) );
set(gca(),  data_bounds , [0, 0; N-1, max_energy+1]);
subplot(4,1,4);
plot2d3(0:Ly-1, energy_y , style=5);
xtitle( Energy of Signals y(n) );
```

```
set(gca(),  data_bounds , [0, 0; Ly-1, max_energy+1]);
```

This is the output of exercise 6:

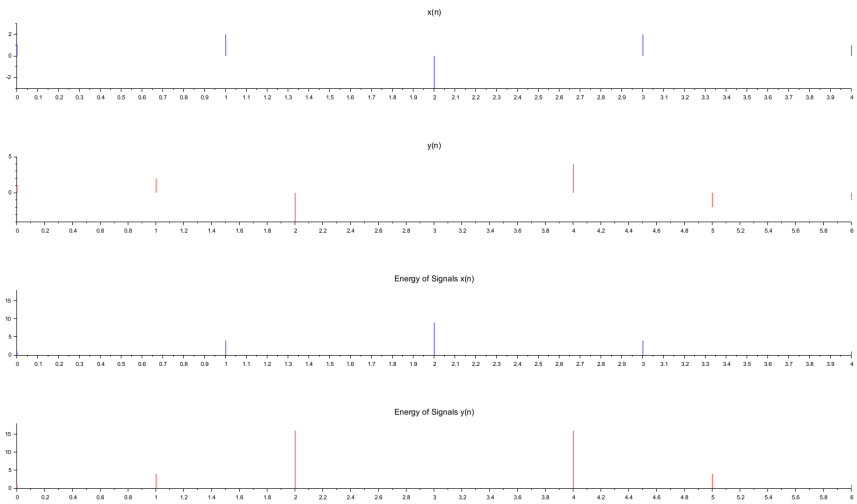

Figure 2: *Exercise 6 Output*

1.5.4 Exercise 7

For a periodic signal $x(n)$ of length N and impulse response $h(n)$ of length M, the circular convolution between $x(n)$ and $h(n)$ is defined as $y(n) = \sum_{k=0}^{M-1} h(k)x[(n-k) \pmod N]$. Explain why $y(n)$ can be deprived that way.

- Compute the convolution $y(n)$ of $x(n) = [\underset{\uparrow}{1}, 2, -3, 2, 1]$ and $h(n) = [\underset{\uparrow}{1}, 0, -1, -1, 1]$ using the "folding and shifting" method. Implement your answer with Scilab script.
- Compute the convolution $y(n)$ of $x(n) = [\underset{\uparrow}{1}, 2, -3, 2, 1]$ and $h(n) = [\underset{\uparrow}{1}, 0, -1, -1, 1]$ using the matrix method. Implement your answer with Scilab script.
- Drawing $x(n), y(n)$ and energy of discrete signals in the same figure.

Solution

For a periodic signal $x(n)$ of length N and impulse response $h(n)$ of length M, the circular convolution between $x(n)$ and $h(n)$ is defined as:

$$y(n) = \sum_{k=0}^{M-1} h(k)x[(n-k) \bmod N]$$

Explanation:

A signal $x(n)$ is periodic with a period $N(N > 0)$ if and only if $x(n+N) = x(n), \forall n$. Circular convolution is used for periodic signals, and the modulo operation in $x[(n-k) \bmod N]$ ensures

that indices wrap around within the signal's period N. This allows the signal to be "folded" and shifted circularly, capturing all interactions between $x(n)$ and $h(n)$ within one period.

Compute the convolution $y(n)$ of:

- $x(n) = [1^\dagger, 2, -3, 2, 1]$
- $h(n) = [1^\dagger, 0, -1, -1, 1]$

Using the "folding and shifting" method:

$$y(n) = \sum_{k=0}^{M-1} h(k)x(n-k)$$

1. The first step is to fold the sequence $x(n)$:

$$x(-n) = [1, 2, -3, 2, 1^\dagger]$$

2. The second step is shifting $x(-k) \to x(-k+n)$. It will shift 5 times due to the period N being 5; after 5 shifts, the signal reverts to its initial value, demonstrating its periodic nature.
3. Compute the convolution for different values of n:
   - With $n = 0$, shift right 0: $x(-k) = [1, 2, -3, 2, 1^\dagger]$.

$$y(0) = h(0) \cdot x(0) + h(1) \cdot x(4) + h(2) \cdot x(3) + h(3) \cdot x(2) + h(4) \cdot x(1) = 4$$

   - With $n = 1$, shift right 1: $x(1-k) = [1, 2, -3, 2^\dagger, 1]$.

$$y(1) = h(0) \cdot x(1) + h(1) \cdot x(0) + h(2) \cdot x(4) + h(3) \cdot x(3) + h(4) \cdot x(2) = -4$$

   - With $n = 2$, shift right 2: $x(2-k) = [1, 2, -3^\dagger, 2, 1]$.

$$y(2) = h(0) \cdot x(2) + h(1) \cdot x(1) + h(2) \cdot x(0) + h(3) \cdot x(4) + h(4) \cdot x(3) = -3$$

   - With $n = 3$, shift right 3: $x(3-k) = [1, 2^\dagger, -3, 2, 1]$.

$$y(3) = h(0) \cdot x(3) + h(1) \cdot x(2) + h(2) \cdot x(1) + h(3) \cdot x(0) + h(4) \cdot x(4) = 0$$

   - With $n = 4$, shift right 4: $x(4-k) = [1^\dagger, 2, -3, 2, 1]$.

$$y(4) = h(0) \cdot x(4) + h(1) \cdot x(3) + h(2) \cdot x(2) + h(3) \cdot x(0) + h(4) \cdot x(0) = 3$$

Thus the output sequence is:

$$y(n) = [4^{\dagger}, -4, -3, 0, 3]$$

Using the matrix method:

- **Step 1:** Construct the circulant matrix X from $x(n)$ where each row n contains $x(n-k) \bmod 5$:

$$X = \begin{bmatrix} 1 & 1 & 2 & -3 & 2 \\ 2 & 1 & 1 & 2 & -3 \\ -3 & 2 & 1 & 1 & 2 \\ 2 & -3 & 2 & 1 & 1 \\ 1 & 2 & -3 & 2 & 1 \end{bmatrix}$$

- **Step 2:** Multiply X by $h = [1, 0, -1, -1, 1]^T$:

$$y = X \cdot h = \begin{bmatrix} 1(1) + 1(0) + 2(-1) + (-3)(-1) + 2(1) \\ 2(1) + 1(0) + 1(-1) + 2(-1) + (-3)(1) \\ -3(1) + 2(0) + 1(-1) + 1(-1) + 2(1) \\ 2(1) + (-3)(0) + 2(-1) + 1(-1) + 1(1) \\ 1(1) + 2(0) + (-3)(-1) + 2(-1) + 1(1) \end{bmatrix} = \begin{bmatrix} 4 \\ -4 \\ -3 \\ 0 \\ 3 \end{bmatrix}$$

Result:

$$y(n) = [4^{\dagger}, -4, -3, 0, 3]$$

This is the Scilab code for Exercise 7:

```
// Folding and Shifting Method
clc;
clear;
close;
x = [1, 2, -3, 2, 1];
h = [1, 0, -1, -1, 1];
N = length(x);
y = zeros(1, N);
for n = 0:N-1
sum_val = 0;
for k = 0:N-1
index = modulo(n - k + N, N);
sum_val = sum_val + h(k+1) * x(index + 1);
end
y(n+1) = sum_val;
end
disp( Folding and Shifting Result: );
disp(y);
// Matrix Method
X = zeros(N, N);
```

```
for n = 0:N-1
for k = 0:N-1
index = modulo(n - k + N, N);
X(n+1, k+1) = x(index + 1);
end
end
y_matrix = X * h';
disp( Matrix Method Result: );
disp(y_matrix);
// Plotting
n = 0:N-1;
energy_x = x.^2;
energy_y = y.^2;
max_e = max([energy_x , energy_y]);


clf();
subplot(4,1,1);
plot2d3(n, x, 5);
xtitle( x(n) );
set(gca(),  data_bounds , [min(n), min(x); max(n), max(x)+1]);
subplot(4,1,2);
plot2d3(n, y, 5);
xtitle( y(n) );
set(gca(),  data_bounds , [min(n), min(y); max(n), max(y)+1]);
subplot(4,1,3);
plot2d3(n, energy_x , 5);
xtitle( Energy of x(n) );
set(gca(),  data_bounds , [min(n), 0; max(n), max_e+1]);
subplot(4,1,4);
plot2d3(n, energy_y , 5);
xtitle( Energy of y(n) );
set(gca(),  data_bounds , [min(n), 0; max(n), max_e+1]);
```

This is the output of Exercise 7:

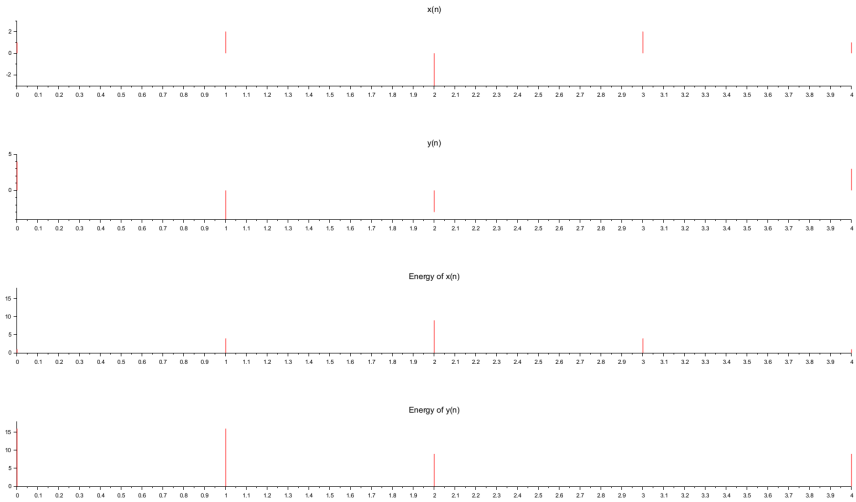

Figure 3: *Output of Exercise 7*

2 Reference

The full source code and this report is available in the GitHub repository: https://github.com/l1ttled1no/DigitalSignalProcessing-Lab